

Dagster at the center of your pipelines

Stop Writing Docs. Let your Pipelines describe themselves. 📖✨

The context

Dagster tried to solve the documentation problem with "Software Defined Assets." WPipe solves it with Auto-Visualizing States. Mermaid Integration: Your pipelines generate their own diagrams.

Why engineers shift to wpipe

- Mermaid Integration: Your pipelines generate their own diagrams.
- Clean Syntax: `@state` defines the intent, WPipe handles the graph.
- Traceability: Know exactly where your data came from and where it's going.

Code-first

Pipelines are Python: versionable, testable, reviewable in Git.

Checkpoints

State persisted to SQLite WAL at every step; resume, don't restart.

Independently lean

<50MB RAM. No broker, no daemon, no metadata server.

What it looks like

```
from wpipe import Pipeline, step

@step(name="step", version="v2.1")
def step(data):
    return {"value": process(data["row"])}

pipe = Pipeline(pipeline_name="dagster_like")
pipe.set_steps([step])

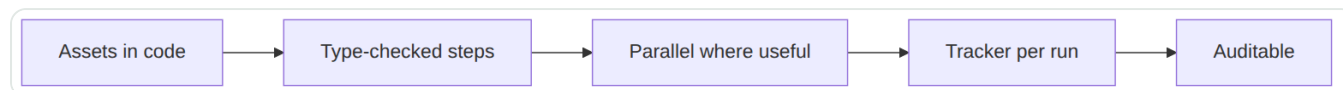
# Step-first resilience without the server.
```

At a glance

Aspect	Legacy / Dagster	wpipe
Definition	Config/DSL or canvas	Python code
State	Scattered / none	SQLite WAL checkpoints
Observability	Partial	Tracker SQL + dashboard
Footprint	Heavy	<50MB RAM

Under the hood

The flow behind the post, as wpipe renders it from code.



+117k downloads

A community of efficiency-first developers.

<50MB RAM

Fits containers, VMs, and Raspberry Pi-class hardware.

SQLite WAL

Industrial durability without a database to administer.

Next step

wpipe is a Python library — try it in your next pipeline:

```
pip install wpipe
```

Steps stay functions you already understand. Docs, diagrams, and dashboards follow. Explore **Dagster** workflows at wpipe.readthedocs.io.

wpipe · Python-native orchestration with SQLite WAL checkpoints, retries, timeouts, parallelism, and self-documenting flows. +117k downloads.